

Actual Code for Part 3

I would just like to make a slight correction that I forgot to mention during our interview. While the algorithm I provided you for part 2 and part 3 is algorithmically correct I didn't include the correct syntax for actual real-time PLC processing. Because the PLC first reads HW I/O, deletes memory, second reads SCL Program, three PLC updates the physical addresses.

I want to make it clear that the names are internal tags and so would not be wiped out by memory. Also it is to be noted as I mentioned earlier that TON Block C can be added in parallel to HMI Valve Activation which turns it into a digital signal from an analog one as mentioned before.

```
SENSOR_A := "3.3 B5 Proximity";  
SENSOR_B := "3.3 B4 Proximity";  
PNEUMATIC_VALVE := "3.3Y1";
```

Also, the pseudo-code I sent you was meant to be an implementation of a state machine. Although I now realize this could be done more compactly by.

CASE HMI_DATA.STATE OF

```
00:  
  IF (HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_A = TRUE) AND  
  (HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_B = TRUE) THEN  
    PNEUMATIC_VALVE := TRUE;  
    HMI_DATA.STATE := 10;  
  ENDIF  
  
10:  
  IF (HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_A = FALSE) AND  
  (HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_B = FALSE) THEN  
    HMI_DATA.VALVE_ACTIVATION := FALSE;  
  
    // Small NOP delay implemented by a TON block A of small time in LL  
  
    HMI_DATA.VALVE_ACTIVATION := TRUE;  
  
    PNEUMATIC_VALVE := TRUE;  
    HMI_DATA.STATE := 20;  
  
  ENDIF
```

20:

```
IF (HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_A = FALSE) AND  
(HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_B = FALSE) THEN
```

```
    PNEUMATIC_VALVE := TRUE;
```

```
    // Small NOP delay implemented by TON Block A
```

```
    PNEUMATIC_VALVE := FALSE;
```

```
    IF SENSOR_B = TRUE THEN
```

```
        // NOP delay implemented by TON Block B
```

```
        IF SENSOR_A = FALSE THEN
```

```
            PNEUMATIC_VALVE := TRUE;
```

```
        ENDIF
```

```
    ENDIF
```

```
    HMI_DATA.STATE := 30;
```

```
ENDIF
```

30:

```
IF (HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_A = TRUE) AND  
(HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_B = FALSE) THEN
```

```
    IF SENSOR_B = FALSE OR SENSOR_A = TRUE THEN
```

```
        // NOP implemented by TON Block B, wait until sensor A becomes FALSE and  
        // sensor B becomes TRUE
```

```
    ENDIF
```

```
    IF SENSOR_B = TRUE AND SENSOR_A = FALSE THEN
```

```
        HMI_DATA.VALVE_ACTIVATION := TRUE;
```

```
        PNEUMATIC_VALVE := TRUE;
```

```
    ENDIF
```

```
    HMI_DATA.STATE := 00;
```

```
ENDIF
```

```
ENDCASE;
```

This forms the state machine $00 \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow 00$. Note that the previous code didn't really need a state machine as it was already implying it.